Scalable Decision Trees and Ensemble Methods

Dr. Robert Loftin

COM6012 Scalable Machine Learning – Spring 2026

(Slides courtesy of Tahsin Khan and Dr. Mauricio Alvarez)

University of Sheffield

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Previously, in COM6012: Exponential Families

A probability density function (PDF) or a probability mass function (PMF) $p(\mathrm{y}|\,\boldsymbol{\theta})$, with $\mathbf{y} \in \mathbb{R}^p$ and $\boldsymbol{\theta} \in \mathbb{R}^d$, is in the exponential family if it is of the form

$$p(\mathbf{y}|\boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} h(\mathbf{y}) \exp\left[\boldsymbol{\theta}^\top \boldsymbol{\mathcal{T}}(\mathbf{y})\right],$$

where $Z(\boldsymbol{\theta}) = \int_y h(\mathbf{y}) \exp\left[\boldsymbol{\theta}^\top \boldsymbol{\mathcal{T}}(\mathbf{y})\right] d\mathbf{y}.$

- $h(\mathbf{y})$ is a scaling constant, often 1.
- $\boldsymbol{\theta}$ are known as the **natural parameters** or **canonical parameters**.
- $\boldsymbol{\mathcal{T}}(\mathbf{y}) \in \mathbb{R}^d$ is called a vector of **sufficient statistics**.
- $Z(\boldsymbol{\theta})$ is known as the **partition function**.

Exponential Family

Distributions in the exponential family can also be expressed as

$$p(\mathbf{y}|\boldsymbol{\theta}) = h(\mathbf{y}) \exp\left[\boldsymbol{\theta}^\top \boldsymbol{\mathcal{T}}(\mathbf{y}) - A(\boldsymbol{\theta})\right],$$

where $A(\boldsymbol{\theta}) = \log Z(\boldsymbol{\theta})$.

- $A(\boldsymbol{\theta})$ is called the **log partition function** or **cumulant function**.
- If $\boldsymbol{\mathcal{T}}(\mathbf{y}) = \mathbf{y}$, we say it is a **natural exponential family**.

Generalised Linear Models

- We want to model the relationship between a response variable y_i, and an input vector $\boldsymbol{x}_i$.
- Consider the case of an unconditional distribution for the response variable

$$p(y_i|\theta, \sigma^2) = \exp\left[\frac{y_i\theta - A(\theta)}{\sigma^2} + c(y_i, \sigma^2)\right],$$

where θ is the natural parameter, $A(\cdot)$ is the log-partition function, σ^2 is the **dispersion parameter**, usually, $\sigma^2 = 1$, and c is the normalization constant.

- From the expression for $p(y_i|\theta, \sigma^2)$ we can see that it corresponds to and exponential family with parameter θ.

Generalised Linear Models

- For example, in logistic regression, θ is the log-odds ratio

$$\theta = \log\left(\frac{\mu}{1-\mu}\right),$$

- where $\mu = \mathbb{E}[y] = p(y = 1)$ is the mean parameter.
- To convert from the mean parameter μ to the natural parameter θ, we can use a function $\psi(\cdot)$, $\theta = \psi(\mu)$.
- $\psi(\cdot)$ is uniquely determined by the form of the exponential family used.
- This mapping is **invertible**, such that $\mu = \psi^{-1}(\theta)$

Link functions

- We now need to relate the parameter θ to
- We first define a linear function of the inputs $\eta_i = \mathbf{w}^\top \mathbf{x}_i$.
- We then define the mean of the distribution as an invertible, monotonic function of this linear combination.
- This function, known as the **mean function**, is denoted by $g^{-1}(\cdot)$, so

$$\mathbb{E}(y_i|\mathbf{x}_i) = \mu_i = g^{-1}(\eta_i) = g^{-1}(\mathbf{w}^\top \mathbf{x}_i).$$

- The inverse of the mean function, $g(\cdot)$, is called the **link function**.

Link Functions

- We are free to choose almost any function we like for $g(\cdot)$, so long as it is invertible, and so long as $g^{-1}(\cdot)$ has the appropriate range.

- For logistic regression, we set $\mu = g^{-1}(\eta_i) = \sigma(\eta_i)$

Relationship Between Functions

$$
\begin{tikzcd}
\boldsymbol{w} \arrow[dr] & & & & \\
& \eta_i \arrow[r, shift left, "g^{-1}"] & \mu_i \arrow[l, shift left, "g"] \arrow[r, shift left, "\Psi"] & \theta_i \arrow[l, shift left, "\Psi^{-1}"] \\
\boldsymbol{x}_i \arrow[ur] & & & &
\end{tikzcd}
$$

$g^{-1}(\cdot)$ is the **mean function**. $g(\cdot)$ is the **link function**.

Canonical Link Functions

- A useful case is where the link function g satisfies $g = \psi$.
- This is called the **canonical link function**.
- In this case $\eta_i = \theta_i = \mathbf{w}^\top \mathbf{x}_i$, so the model becomes

$$p(y_i|\mathbf{x}_i, \mathbf{w}, \sigma^2) = \exp\left[\frac{y_i \mathbf{w}^\top \mathbf{x}_i - A(\mathbf{w}^\top \mathbf{x}_i)}{\sigma^2} + c(y_i, \sigma^2)\right].$$

Canonical Link functions

Distribution	**Link** $g(\mu)$	$\theta = \psi(\mu)$	$\mu = \psi^{-1}(\theta)$
$\mathcal{N}(\mu, \sigma^2)$	identity	$\theta = \mu$	$\mu = \theta$
$\mathrm{Ber}(\mu)$	logit	$\theta = \log(\frac{\mu}{1-\mu})$	$\mu = \sigma(\theta)$
$\mathrm{Poi}(\mu)$	log	$\theta = \log(\mu)$	$\mu = \exp(\theta)$
$\mathrm{Ga}(a, b)$	inverse	$\theta = \mu^{-1}$	$\mu = \theta^{-1}$

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Decision Trees

- Decision tree learning refers to a class of supervised machine learning algorithms that build decision tree models based on labelled data.

- Nodes in a decision tree are either:
  - **Interior nodes** (or **decision nodes**) – which specify tests to be carried out on one for the **descriptive features** of a query.
  - **Leaf nodes** – which specify the predictions for queries reaching these nodes (either a **predicted class label** or a **predicted regression value**).

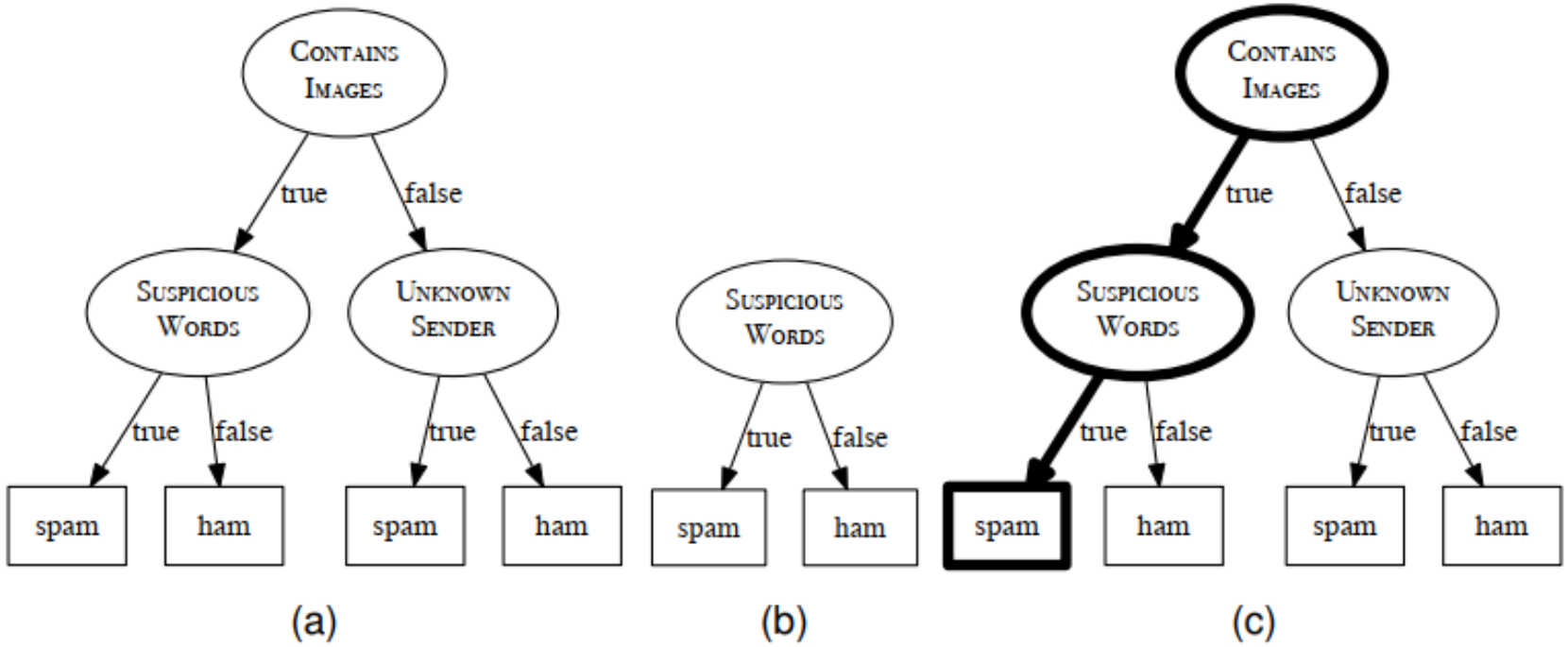
- Spark supports decision trees for both classification and regression, with binary, categorical, and continuous features.
  - All decision trees in Spark are **binary**, with two children for every interior node.

Binary Classification: Spam Prediction

An email spam prediction dataset.

ID	Suspicious Words	Unknown Sender	Contains Images	Class
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Two Decision Trees and a Query Instance

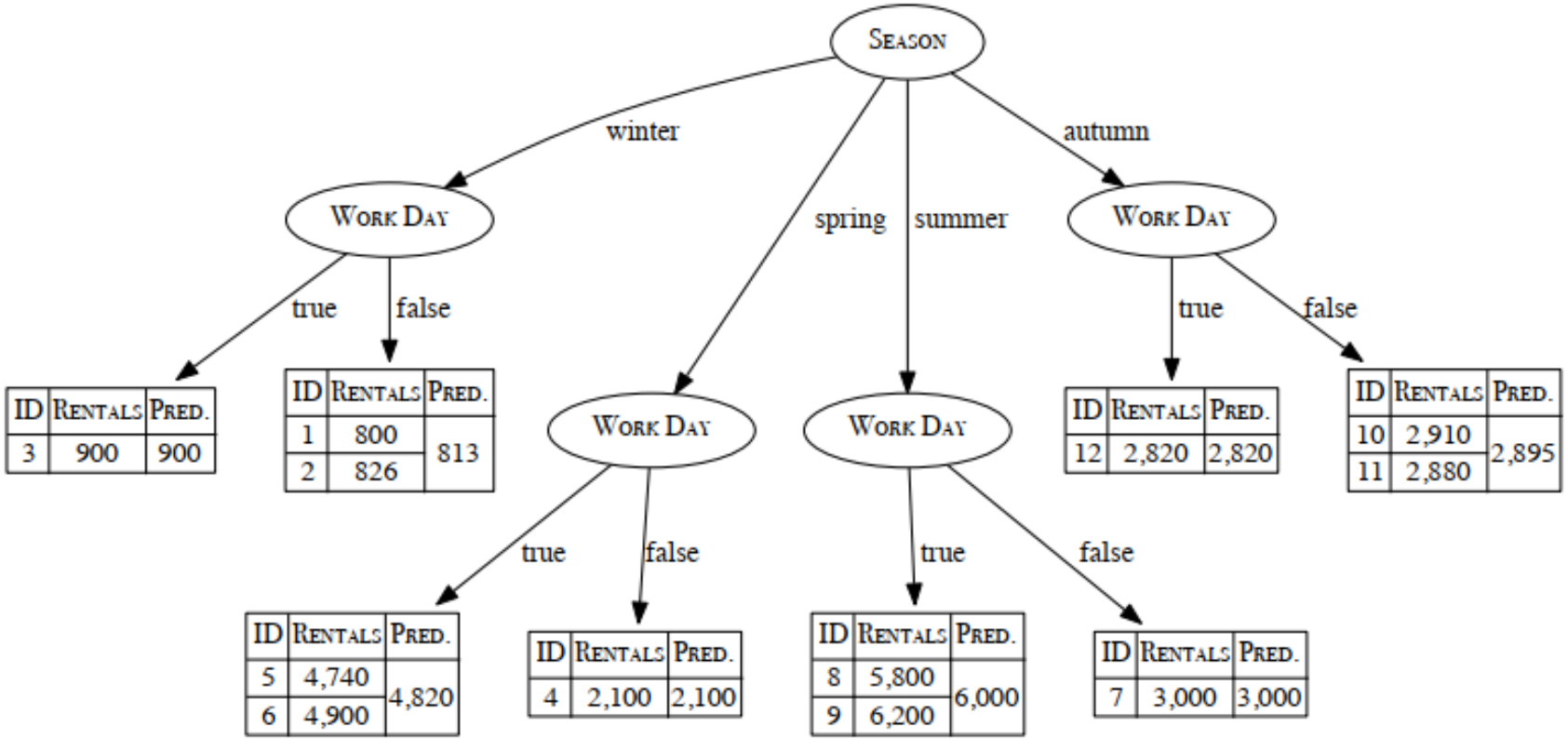


(a) and (b) show two decision trees that are consistent with the instances in the spam dataset. (c) shows the path taken through the tree shown in (a) to make a prediction for the query instance: Suspicious Words = *'true'*, Unknown Sender = *'true'*, Contains Images = *'true'*.

Regression: Bike Rentals Per Day

ID	Season	Work Day	Rentals
1	winter	false	800
2	winter	false	826
3	winter	true	900
4	spring	false	2 100
5	spring	true	4 740
6	spring	true	4 900
7	summer	false	3 000
8	summer	true	5 800
9	summer	true	6 200
10	autumn	false	2 910
11	autumn	false	2 880
12	autumn	true	2 820

Regression Tree



The final decision tree induced from the dataset. This tree lists the instances that ended up at each leaf node and the prediction (PRED.) made by each leaf node.

Advantages of Decision Trees

- Compared to GLMs:
  - Handle categorical features well
  - Extend naturally to the multiclass classification setting
  - Do not require feature scaling
  - Able to capture non-linearities and feature interactions
- **Easy for humans to interpret**

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Binary Decision Trees

- Spark only builds binary decision trees – each interior node has exactly two child nodes.
- Trees are built recursively starting from the root node, with each interior node corresponding to a binary partition of the training data
- At each interior node, Spark identifies the best **split**, that is the best rule for partitioning the data at that node.
- Each split depends on only a single descriptive feature – a single categorical or continuous features can allow multiple splits.

Choosing the Best Split

- The best split at a decision node n maximizes the **information gain**

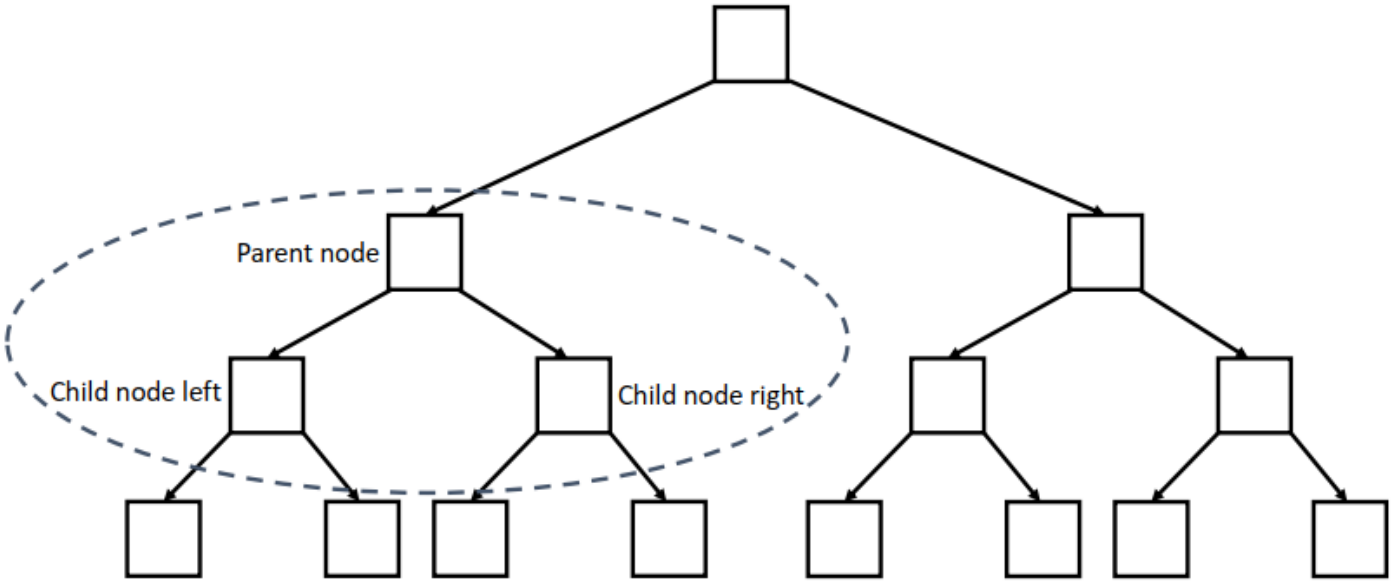
- The best split s^* is at node n is chosen from set S_n as

$$s^* \in \underset{s \in S_n}{\operatorname{argmax}}\, IG(D_n, s)$$

- Here, $IG(D_n, s)$ is the information gain achieved when split s is applied to partition D_n of the dataset D (for the root node $D_n = D$).

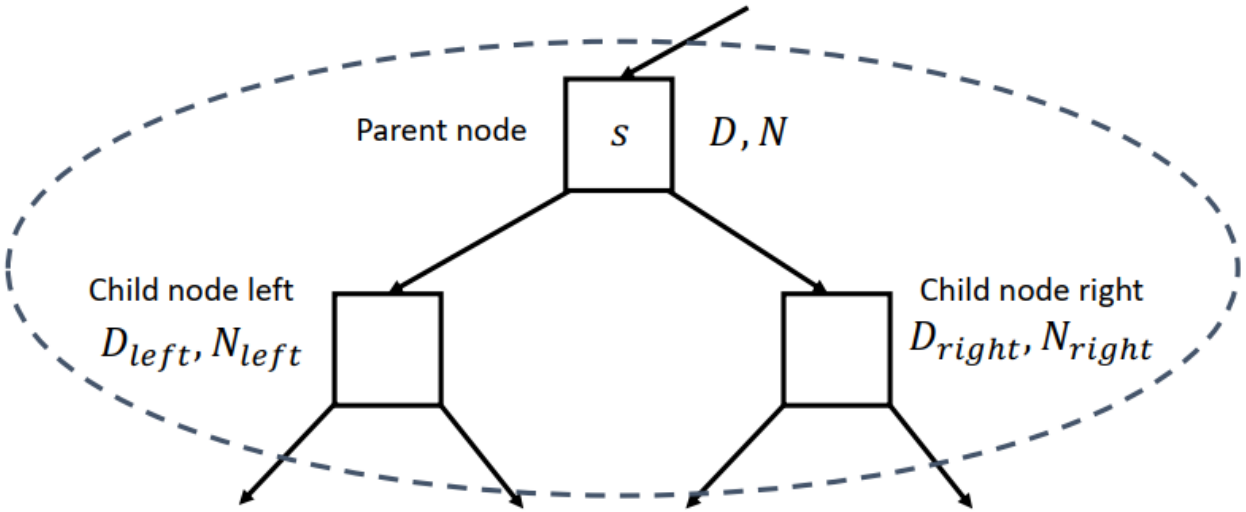
Information gain

- The information gain is the difference between the **impurity** of the parent node impurity and average **impurity** of the two child nodes.



Information gain

- Assuming, that a split $\boldsymbol{s}$ partitions the dataset D of size N into two datasets D_{left} and D_{right} of size N_{left} and N_{right}



- The information gain is computed as:

$$IG(D,s) = Impurity(D) - \left(\frac{N_{left}}{N}\ Impurity\left(D_{left}\right) + \frac{N_{right}}{N}\ \text{Impurity}\left(D_{right}\right)\right)$$

Node impurity

- The impurity $Impurity(D)$ of a dataset D measures the homogeneity of the class labels or regression values in D.
- The table below shows the impurity measures currently implemented for decision trees in Spark:

Impurity	Task	Formula	Description
Gini impurity	Classification	$\sum_{i=1}^{C} f_i(1-f_i)$	f_i is the frequency of label i at a node and C is the number of unique labels.
Entropy	Classification	$\sum_{i=1}^{C} -f_i log(f_i)$	f_i is the frequency of label i at a node and C is the number of unique labels.
Variance	Regression	$\frac{1}{N}\sum_{i=1}^{N}(y_i-\mu)^2$	y_i is label for an instance, N is the number of instances and μ is the mean given by $\frac{1}{N}\sum_{i=1}^{N} y_i$.

*frequency refers to probability

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Categorical Features

- For a categorical feature with M possible (unordered) values, a split will need to partition D into two sets based on this feature.
- There are $2^{M-1} - 1$ possible partitions of the M values into two groups.
- In general, Spark will consider all $2^{M-1} - 1$ possible partitions, which can be computationally prohibitive.
- If $2^{M-1} - 1 \geq$ `maxBins`, then Spark applies a heuristic in which Spark orders values by the impurity of the subset of D that takes on each value, and only considers the $M - 1$ splits defined by this ordering.

Categorical Features: Binary Classification

- For the special case of binary classification, the number of partitions can be reduced to $M - 1$ by ordering the categorical feature values by proportion falling in the outcome class 1

- The table below gives the proportion of all positive instances in the dataset that take on each of the values for the "season" feature.

Categorical value	Proportion
spring	0.2
summer	0.3
autumn	0.05
winter	0.45

Categorical Features: Binary Classification

- Order the categorical values by proportion, from least to greatest:
  - `[autumn, spring, summer, winter]`
  - The number possible partitions is $M - 1 = 3$

Option	Group 1	Group 2
1	autumn	spring, summer, winter
2	autumn, spring	summer, winter
3	autumn, spring, summer	winter

- We have transformed an unordered feature into an ordered feature.
- We can show that this gives the **optimal split** for both the entropy and Gini impurities, among all possible $2^{M-1} - 1$ splits

Categorical Features: Regression

- A similar result holds for regression problems, where we order categorical values by the This result also holds for a regression problem when using the square error loss.

- For regression tasks, the values are ordered by the increasing mean prediction given each value, again yielding $M - 1$ partitions.

- More details on the theoretical results can be found in book *The Elements of Statistical Learning*, 2nd edition, section 9.2.4

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Continuous Features

- For continuous features, splits are defined by a `threshold`

if $x_i \leq$ `threshold` **then**
    Move to the left/right branch
else
    Move to the right/left branch
end if

- When working with small datasets, we can directly use the values of the feature for each instance as a candidate split threshold.
- This is not feasible for large, distributed datasets.

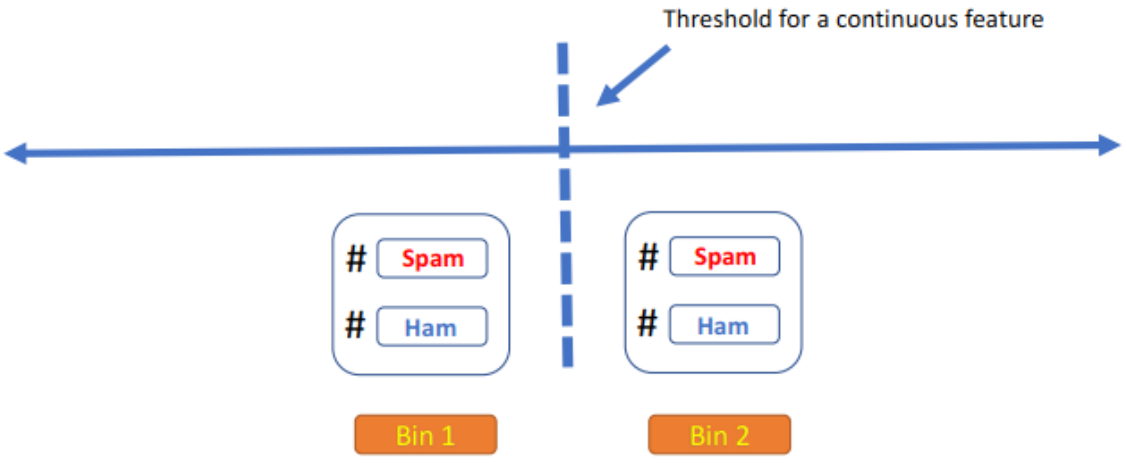
Handling continuous features

- Spark's implementation computes an approximate set of split candidates by computing the `maxBins` quantiles of a representative sample of the data.

- The `maxBins` parameter can be set as needed to balance computational efficiency with the quality of the resulting tree.

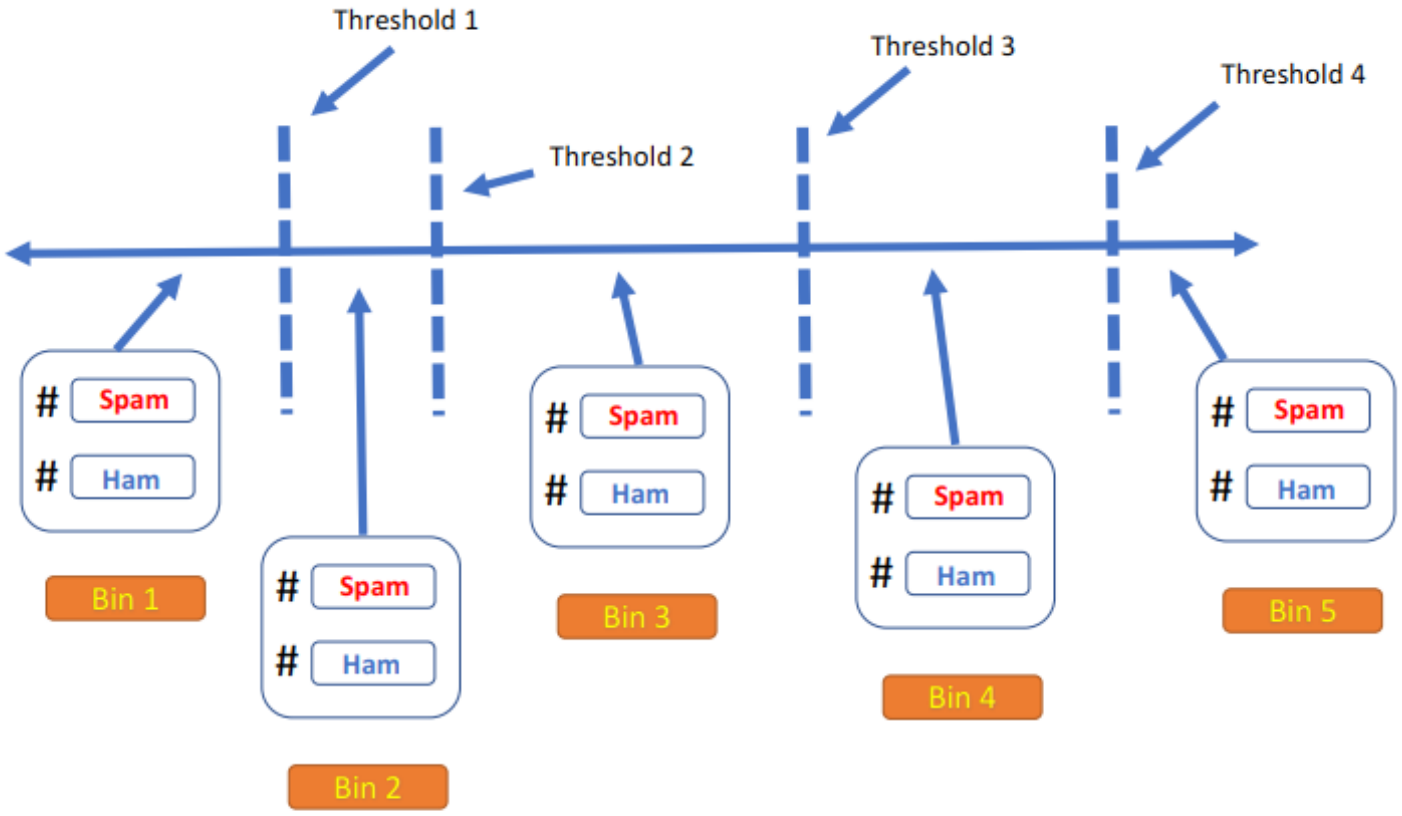
- Once the "bins" have been constructed, each instance in D must be assigned to one such bin to compute the impurity of each of the `maxBins` $- 1$ possible partitions.

Bins for Continuous Features



Bins in continuous features

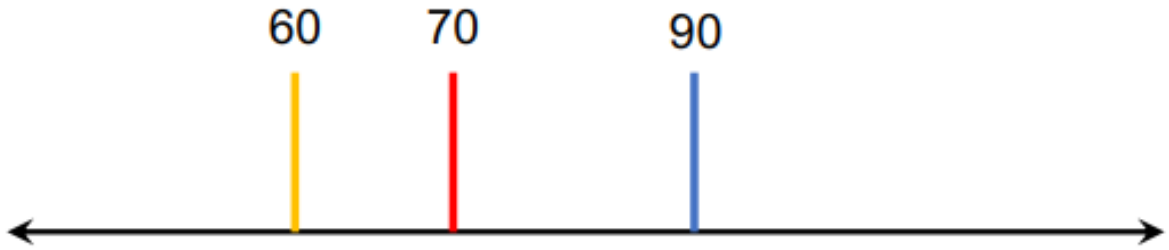
Bins for Continuous Features



Bins in continuous features

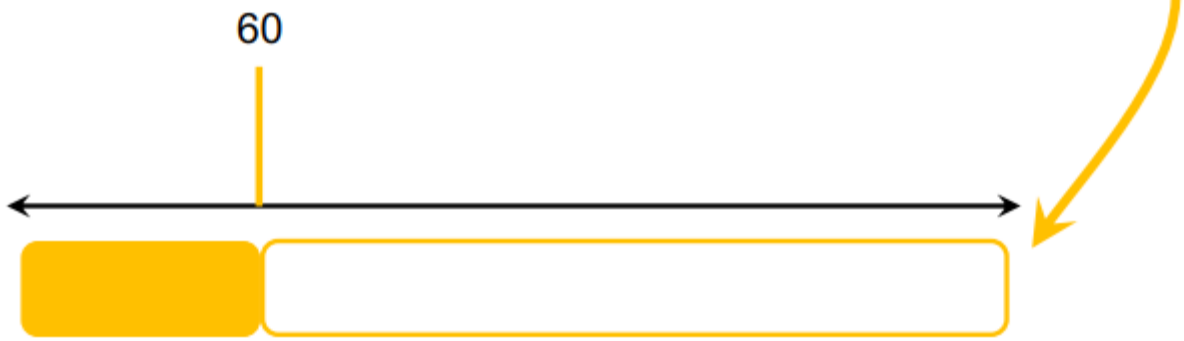
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



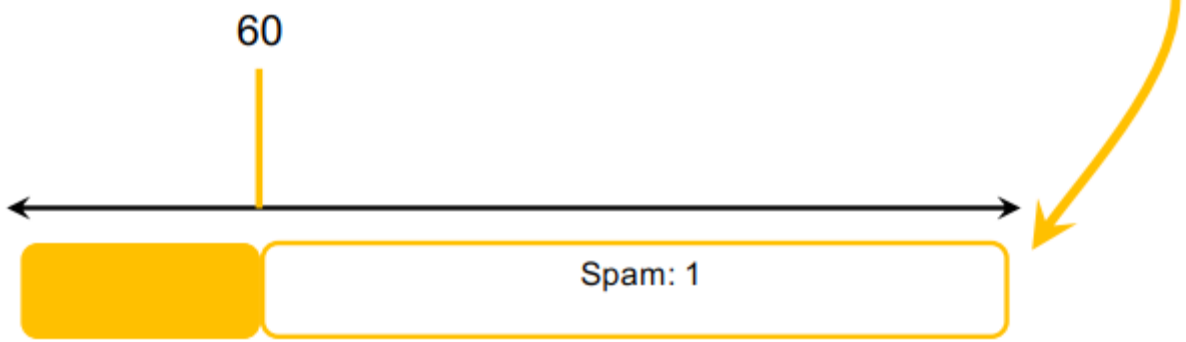
Why bins? Naïve approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



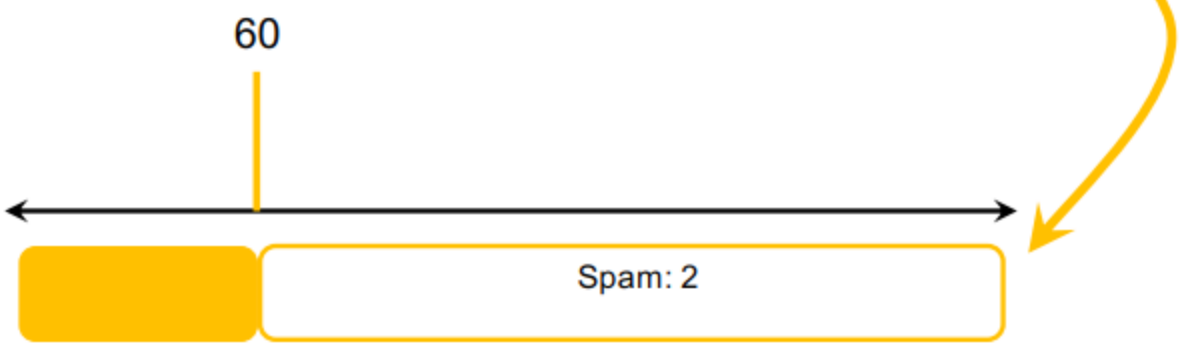
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



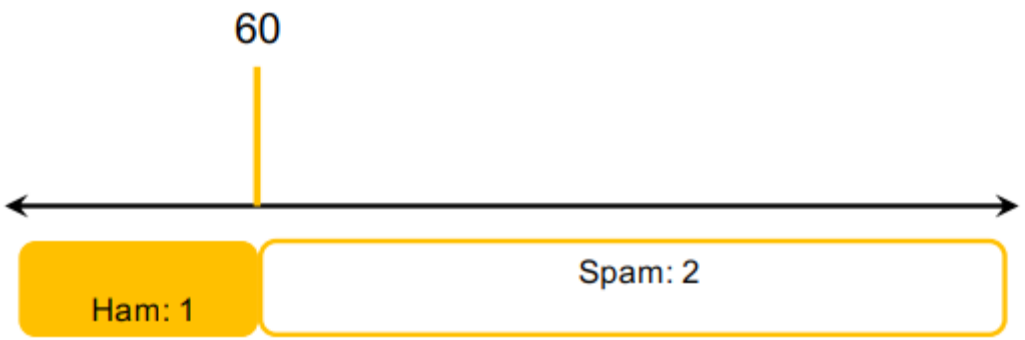
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



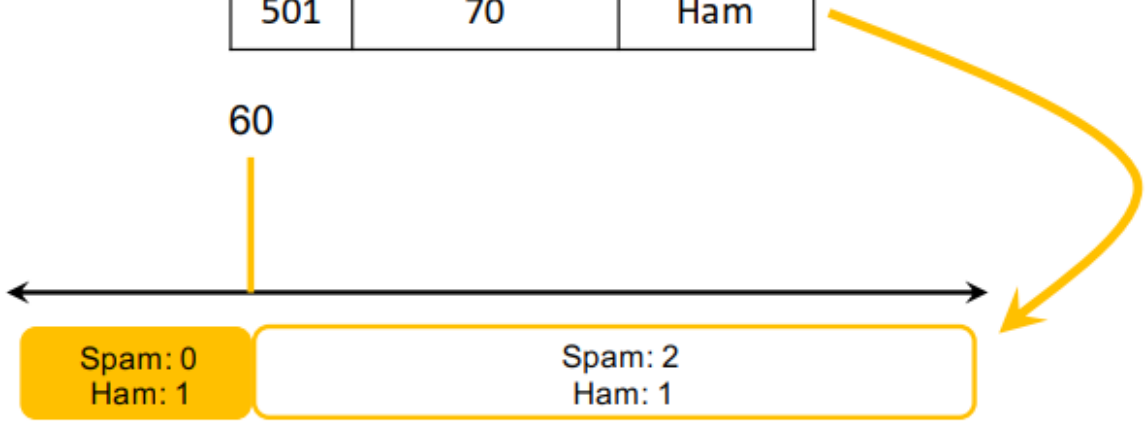
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



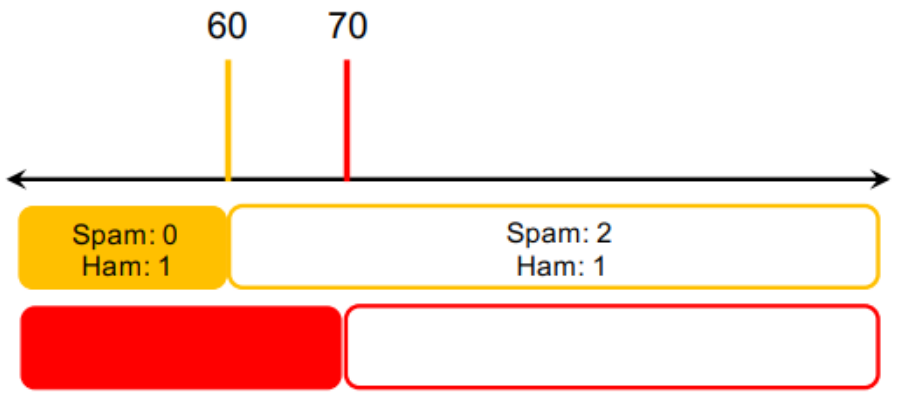
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



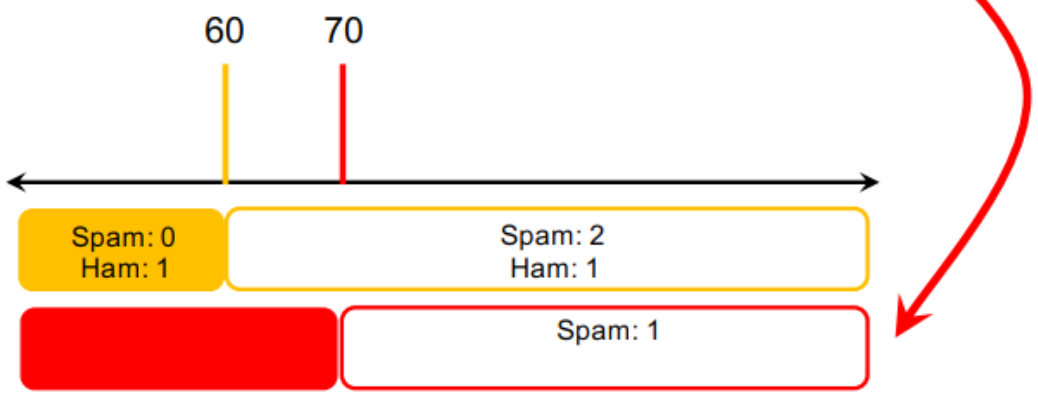
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



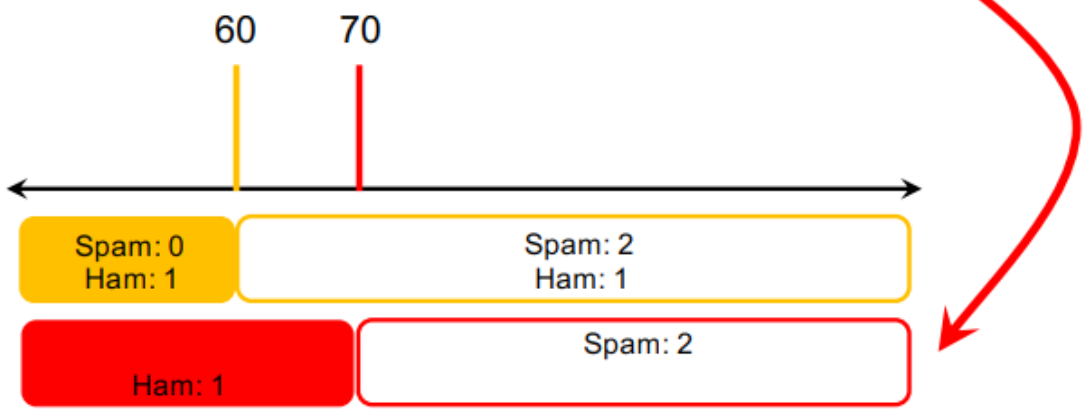
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



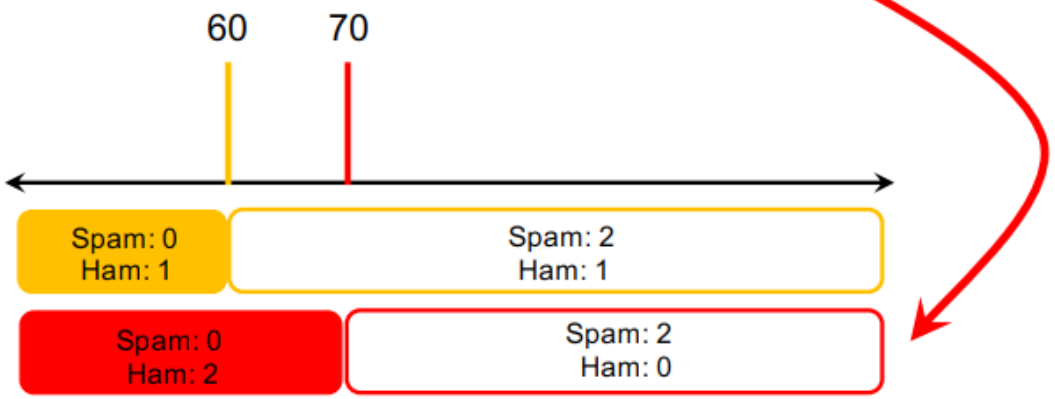
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



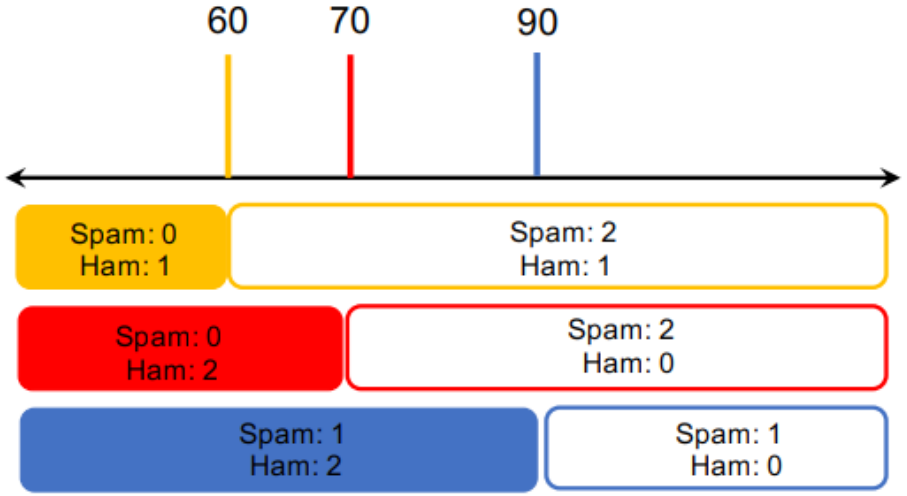
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



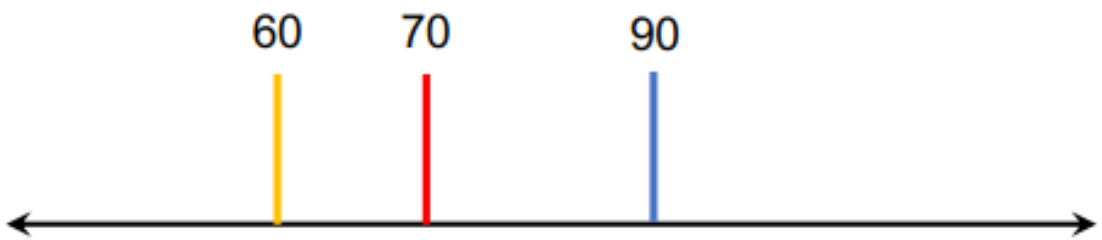
Why Bins? Naïve Approach

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



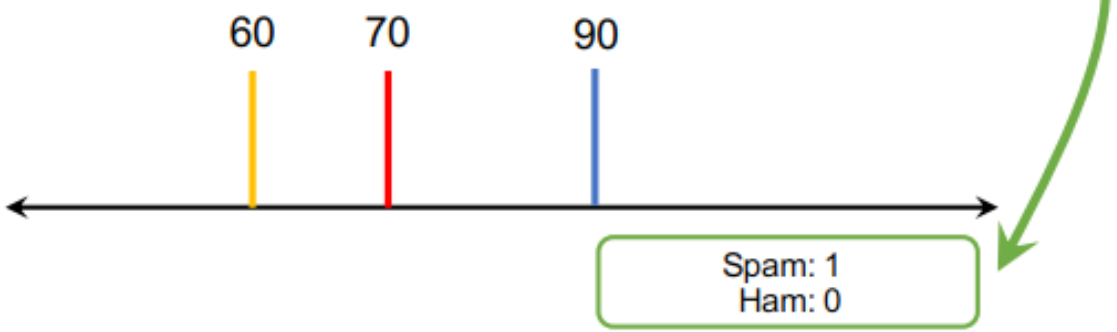
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



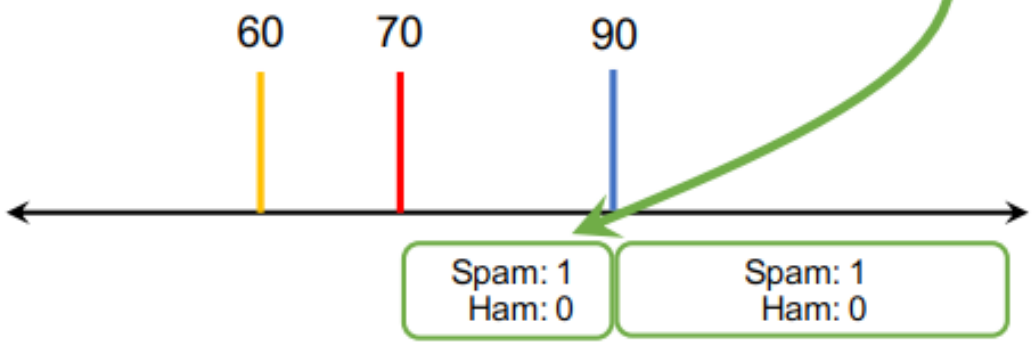
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



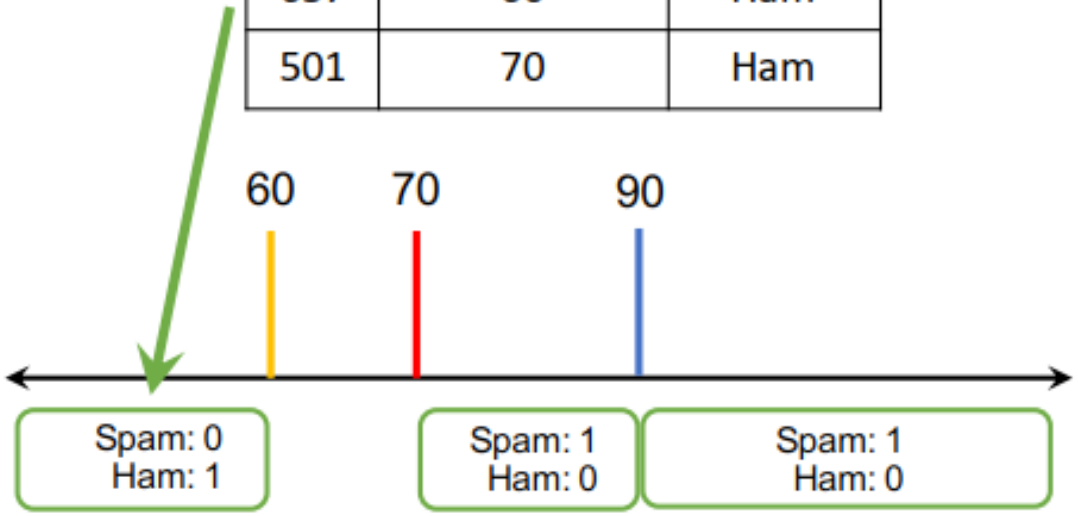
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



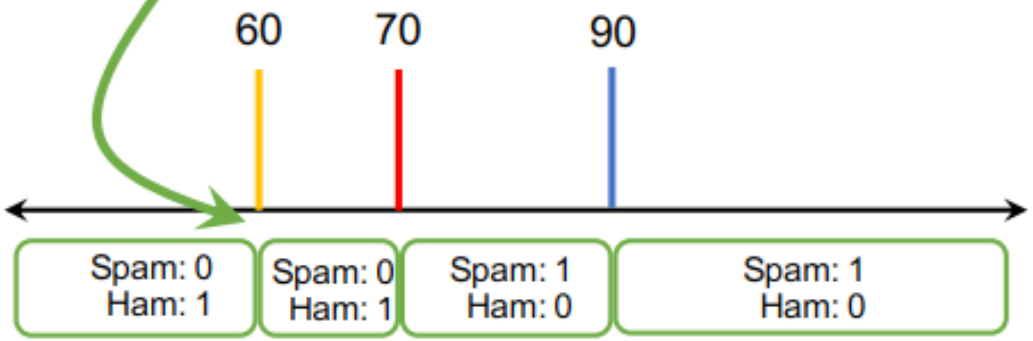
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



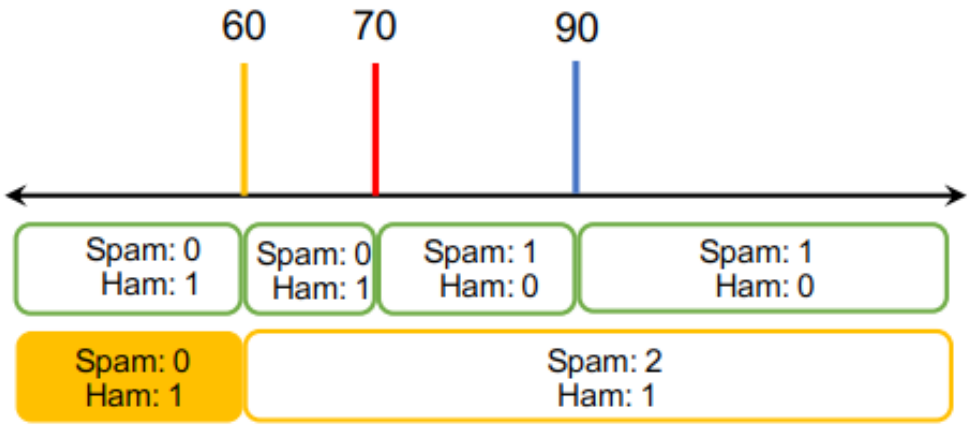
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



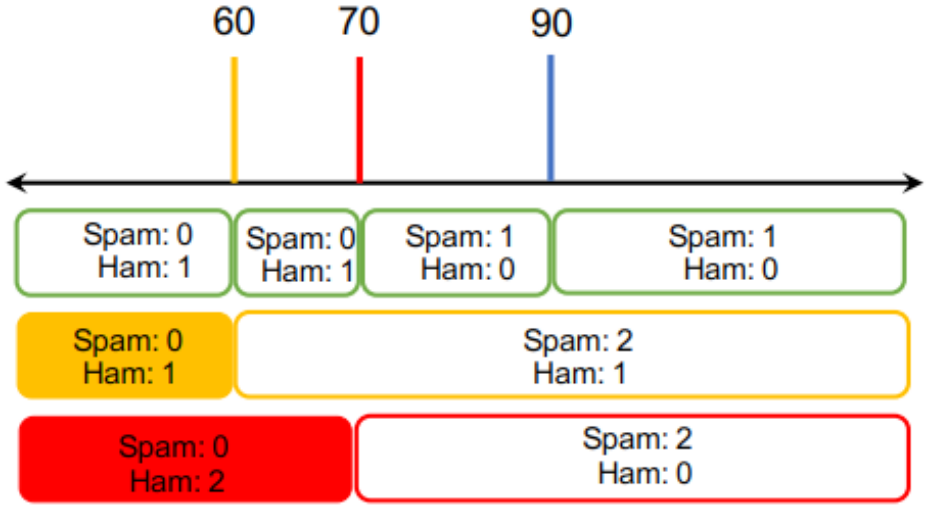
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



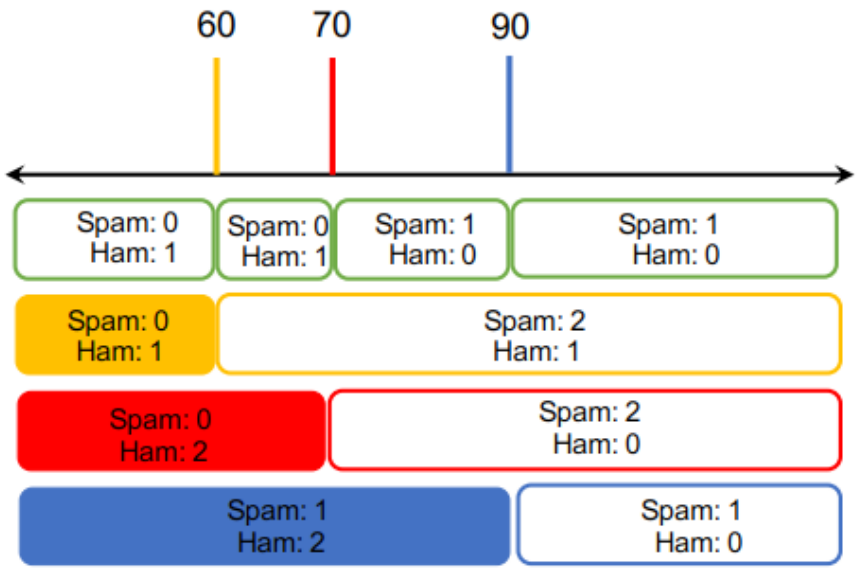
Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



Why Bins? Binary Search

ID	Frequency of word “free”	Class
908	95	Spam
345	90	Spam
657	60	Ham
501	70	Ham



What is the Advantage of Using Bins?

- Assume there are *m* thresholds. With the naïve approach, for each instance in D, we need to perform *m* operations.

- Using binning, it is possible to use binary search for each instance to identify the bin it belongs to.

- Binary search takes $\log(m)$ operations.

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

PLANET: Distributed Decision Tree Learning

- PLANET is an algorithm for learning decision trees efficiently on extremely large datasets, by parallelising as much work as possible.

- The full dataset D is partitioned into subsets D_j, with the instances in D_j being processed by worker j (this is called **horizontal partitioning**).

- When D is large, the most expensive operation is computing the impurity of D, and of the partitions D_{left} and D_{right} for each split s.

- PLANET distributes the computation of impurities for all split candidates across multiple workers, while the central controller selects the best split.

PLANET: Distributed Impurity Computation

- Define functions g and f such that

$$IG(D, s) = f\left(\sum_{(x,y)\in D} g(x, y, s)\right)$$

- The functions g and f depend on the choice of impurity function.
- $g: X \times Y \times S \to \mathbb{R}^c$ computes c **sufficient statistics** of $IG(D, s)$
- $f: \mathbb{R}^c \to \mathbb{R}$ computes $IG(D, s)$ using these statistics

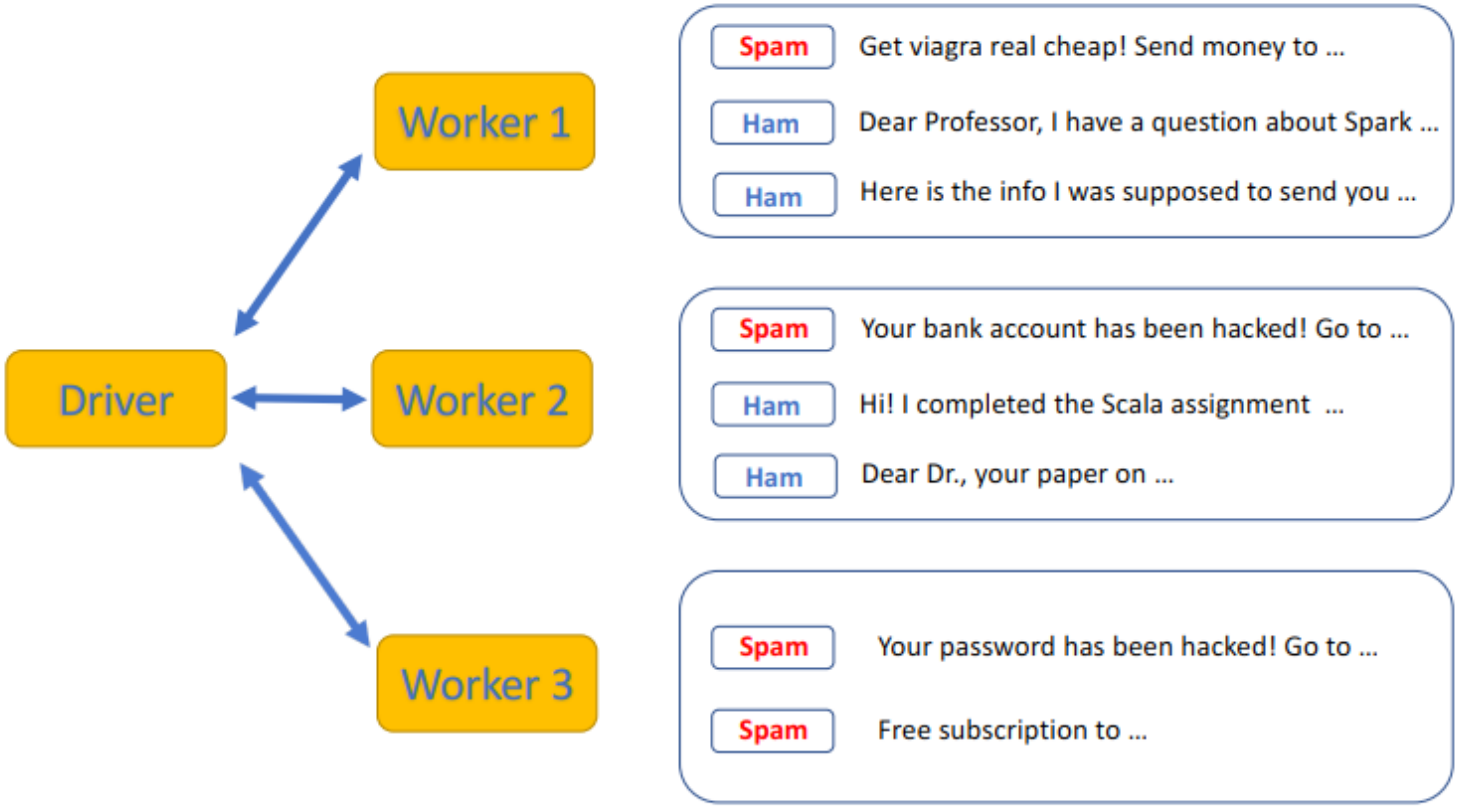
PLANET: Distributed Impurity Computation

- Each worker j computes $g_j(s) = \sum_{(x,y)\in D_j} g(x, y, s)$ for each $s \in S$
- The controller then computes the best s^* split as

$$s^* \in \operatorname*{argmax}_{s\in S} f\left(\sum_j g_j(s)\right)$$

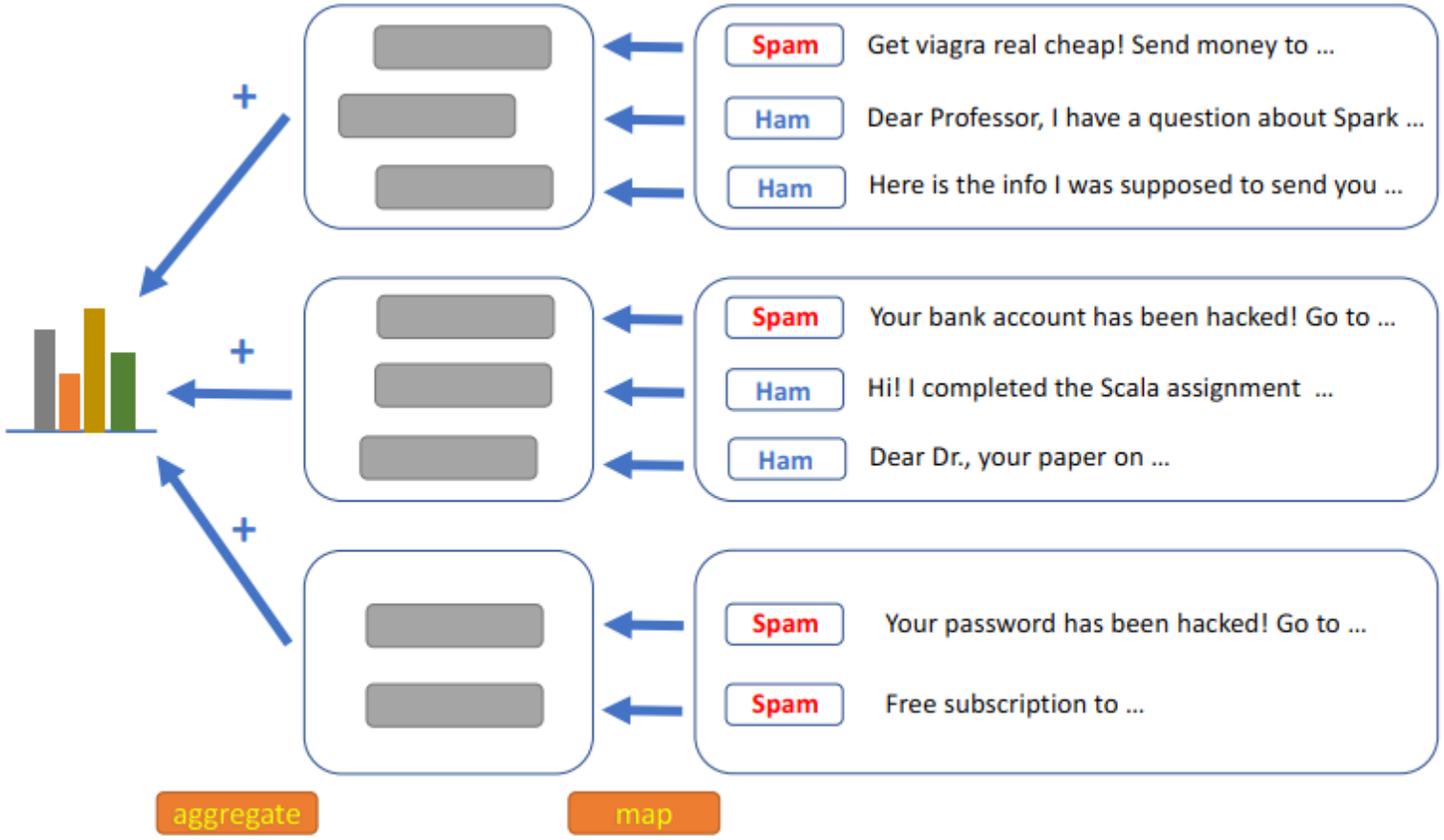
- What are these sufficient statistics $g_j(s) \in \mathbb{R}^c$?
  - For binary classification these are the label counts for both sides of the split s, such that $c = 4$ (two statistics for branch).
  - For regression these are the count, the sum, and the squared sum ($c = 6$)

The Distributed Dataset



A distributed dataset

The Distributed Dataset



A distributed dataset

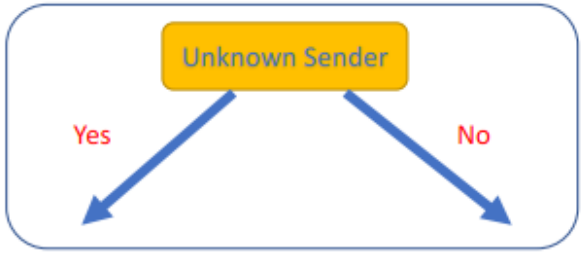
PLANET: Parallelism Across Nodes

- Let p be the number of descriptive features. If we allow at most B splits per feature, then we need only consider pB splits per node.
- Each instance in D can only belong to a single node at depth t, we need only apply g to each instance pB to compute the statistics for all nodes at depth t.
- From $t = 0$ the tree can be built incrementally, choosing splits for all nodes at level t via a single round of communication between the workers and controller.
- The total computation performed by the workers is $O(pBd|D|)$, where d is the depth of the tree, but this can be parallelised across the workers.

PLANET: Communication Cost

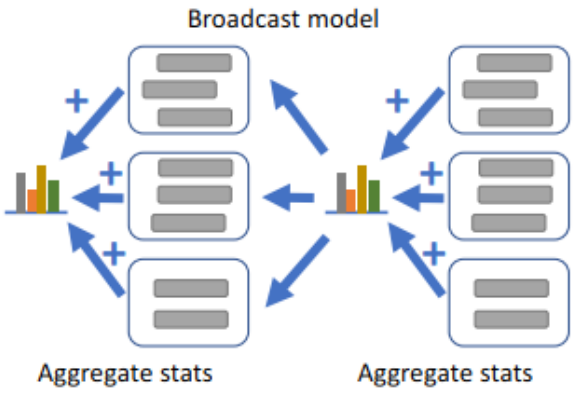
- There is a substantial communication cost, however, as for each level t the k workers must send pBk sets of c sufficient statistics for each node.
- As there are potentially $2^d - 1$ decision nodes in the tree, the total communication complexity will be $O\left(2^d pBkc\right)$.
- Communication costs are therefore exponential in d.

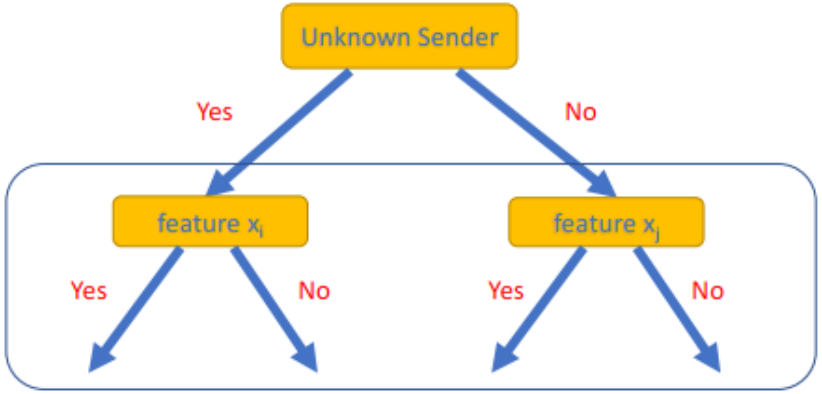
Building a Decision Tree



Building the tree in Spark

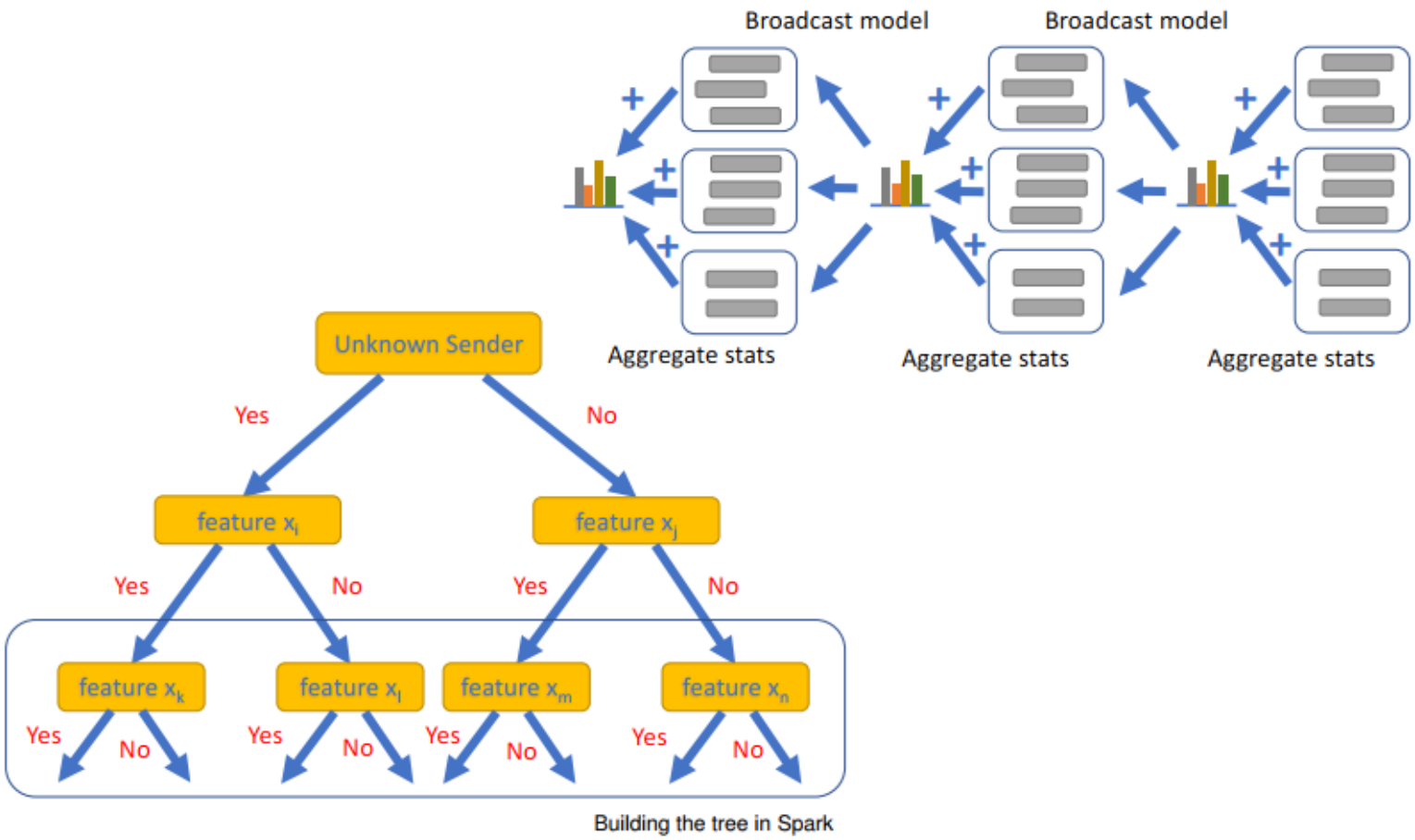
Building a Decision Tree





Building the tree in Spark

Building a Decision Tree



Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

PySpark API for Decision Trees

- PySpark uses two different classes for classification and regression:
  - For classification: `DecisionTreeClassifier`
  - For regression: `DecisionTreeRegressor`

class `pyspark.ml.classification.`**`DecisionTreeClassifier`**(*featuresCol='features', labelCol='label', predictionCol='prediction', probabilityCol='probability', rawPredictionCol='rawPrediction', maxDepth=5, maxBins=32, minInstancesPerNode=1, minInfoGain=0.0, maxMemoryInMB=256, cacheNodeIds=False, checkpointInterval=10, impurity='gini', seed=None*)

class `pyspark.ml.regression.`**`DecisionTreeRegressor`**(*featuresCol='features', labelCol='label', predictionCol='prediction', maxDepth=5, maxBins=32, minInstancesPerNode=1, minInfoGain=0.0, maxMemoryInMB=256, cacheNodeIds=False, checkpointInterval=10, impurity='variance', seed=None, varianceCol=None, weightCol=None, leafCol='', minWeightFractionPerNode=0.0*) [source]

Adjustable Parameters

- `maxDepth` – the maximum depth of a tree.

- `maxBins` – the maximum number of bins for discretizing continuous features, and the maximum number of splits considered for categorical features. Must be ≥ 2 and $\geq$ number of categories for any feature.

- `impurity` – the impurity function used for information gain calculation.
  - `"gini" or "entropy"` for classification (defaults to `"gini"`)
  - `"variance"` for regression

- Several other parameters will be explored in the lab session.

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Ensemble Methods

- Rather than creating a single model, ensemble methods make predictions by aggregating the outputs of a collection of models.
  - For classification, we predict the class chose by a (weighted) majority.
  - For regression, we take the (weighted) average of these predictions.
- We refer to a set of models as an **ensemble**. Ensemble methods can be applied on top of a variety of base learning algorithms.
- For ensemble methods to work, each model must be different from the others in some way.
- Here we discuss two common ways of creating diverse ensembles: **Bagging** and **Boosting**.

Bagging

- In **bagging** (short for **bootstrap aggregating**) each model is trained on a random sample drawn from the data.
  - This sample is known as a **bootstrap sample**.
  - Each bootstrap sample is **sampled with replacement** from the dataset.

- Even if a bootstrap sample is as large as the original dataset, with high probability it will leave out some of the original instances. So:
  - Each bootstrap sample will be different.
  - Models trained on different bootstrap samples will also be different.
  - Any outlier in the data will only affect a subset of models. As a result, bagging tends to reduce **overfitting** as the ensemble grows.

Boosting

- **Boosting** algorithms build the ensemble iteratively, learning each new model based on the errors made by the previous models.

- Each new model added to the ensemble learns to correct some of the errors made by the previous models on the training data.

- Unlike Bagging methods, Boosting algorithms can be more likely to overfit as the ensemble gets larger.
  - New models may focus on correctly predicting outliers in the data

AdaBoost

- The **AdaBoost** algorithm **reweights** the training data to bias new models to focus on instances that were previously misclassified.

- AdaBoost requires that the underlying algorithm accepts such weights. Otherwise, we resample the dataset proportionally to these weights, creating a **replicated training set**.

- The weight w_i for each instance i is initialized uniformly as $\mathrm{w}_i = \frac{1}{n}$, where n is the number of instances in the training dataset.

- Weights are increased for instances that were incorrectly predicted by the latest model in the ensemble and decreased for instances that were correctly predicted by this model.

AdaBoost

- For binary classification, at each iteration t the AdaBoost algorithm learns a model f_t under the current weights and then:
  1. Computes the weighted error $\epsilon_t = \frac{\sum_{i=1}^{n} w_i 1\{y_i \neq f_t(x_i)\}}{\sum_{i=1}^{n} w_i}$ of the model f_t.
  2. Updates the weights for instances misclassified by f_t as $w_i \leftarrow w_i/2\epsilon_t$, and for correctly classified instances as $w_i \leftarrow w_i/2(1-\epsilon_t)$. This increases the relative weight of misclassified instances.
  3. Calculates a confidence factor $\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$ for model f_t which increases as ϵ_t decreases.

- If $\sum_{t=1}^{T} \alpha_t f_t(x) \geq \frac{1}{2}\sum_{t=1}^{T} \alpha_t$ then the ensemble predicts a class of 1 for input x, and predicts a class of 0 otherwise.

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Random Forest - Bagging for Decision Trees

- The **Random Forest** algorithm applies **bagging** to construct an ensemble of decision trees.

- In addition to bagging, at each decision node the Random Forest algorithm only considers a random subset of features to split on.
  - This is an instance of **random subspace sampling**.
  - This prevents the trees from all splitting on the same set of features.

- As with other bagging methods, Random Forest becomes less likely to overfit as the size of the ensemble grows.

Random Forest – PySpark API

- Spark supports Random Forests for multiclass classification and regression, and uses the existing Spark decision tree implementation:
  - `RandomForestClassifier` – For classification
  - `RandomForestRegressor` – For regression

- In addition to the base parameters for decision tree learning, key parameters to adjust for Random Forests include:
  - `numTrees` – the number of trees in the random forest
  - `subsamplineRate` – controls the size of the bootstrap samples
  - `featureSubsetStrategy` – controls the subspace sampling process

Contents

- Recap of Generalised Linear Models
- Review of Decision Trees
- Decision Trees in Apache Spark
  - Available **impurity measures**
  - Split candidates for **categorical features**
  - Split candidates for **continuous features**
  - The PLANET algorithm for distributed decision tree learning
  - `spark.ml` API for decision trees
- Ensemble methods for Decision Trees in Spark
  - Bagging and Boosting
  - Random Forests
  - Gradient-Boosted Trees

Gradient Boosted Trees

- Spark also supports Gradient-Boosted Trees (GBTs), boosting for decision trees, using the existing regression tree implementation.

- Unlike AdaBoost, GBTs **relabel** each instance x_i in the training data with the pseudo-residuals $r_i = -\frac{\partial}{\partial F_t(x_i)} l\big(y_i, F_t(x_i)\big)$, where $F_t(x_i)$ is the prediction of the current ensemble, defined as

$$F_t(x) = \sum_{k=0}^{t} \gamma_t f_t(x)$$

- GBTs fit a regression tree f_{t+1} to this relabelled data.

Gradient-Boosted Trees – PySpark API

- PySpark implements GBTs for regression and binary classification:
  - `GBTClassifier` – For binary classification
  - `GBTRegressor` – For regression
- Spark GBTs do not yet support multiclass classification
- Spark GBTs also allow for bagging and subspace sampling
- Additional parameters that can be adjusted include:
  - `maxIter` – the maximum size of the GBT ensemble
  - `lossType` – for regression, can choose between the squarederror and absolute error losses. Only the logistic loss is available for classification.

References

- PySpark Website: Spark API documentation
- Book: Fundamentals of Machine Learning for Predictive Analytics by Kelleher, Mac Namee and D'Arcy, 2015
  - *available online through the University of Sheffield library*
- YouTube videos about Spark Decision Trees:
  - *Scalable Decision Trees in Spark MLlib* by Manish Amde (contributor to the MLlib implementation of Decision Trees in Spark)
  - *Decision Trees on Spark* by Joseph Bradley (Databricks)
- References for PLANET
  - **Original Paper:** *PLANET: Massively Parallel Learning of Tree Ensembles with MapReduce* by Panda et al. (2009)
  - **Simpler Explanation:** *Yggdrasil: An Optimized System for Training Deep Decision Trees at Scale* by Abuzaid et al. (2016)